

Práctica 2: Logging System

Desarrollo en la Nube · Prof. Marcela Rosales · ITESO

Equipo 3

Integrantes Luis Guillermo Hernandez Casillas · Carlos Emiliano Olmedo Navarro · Isaian Ayala García

Fecha 1 de octubre de 2026

Repositorio <https://github.com/luishdezcnube.git>

Video <https://youtu.be/SFruylpXJo0>

DESCRIPCIÓN DEL PROYECTO

El sistema ingiere logs de servidores OpenSSH, los clasifica y los expone para consulta, usando únicamente servicios administrados.

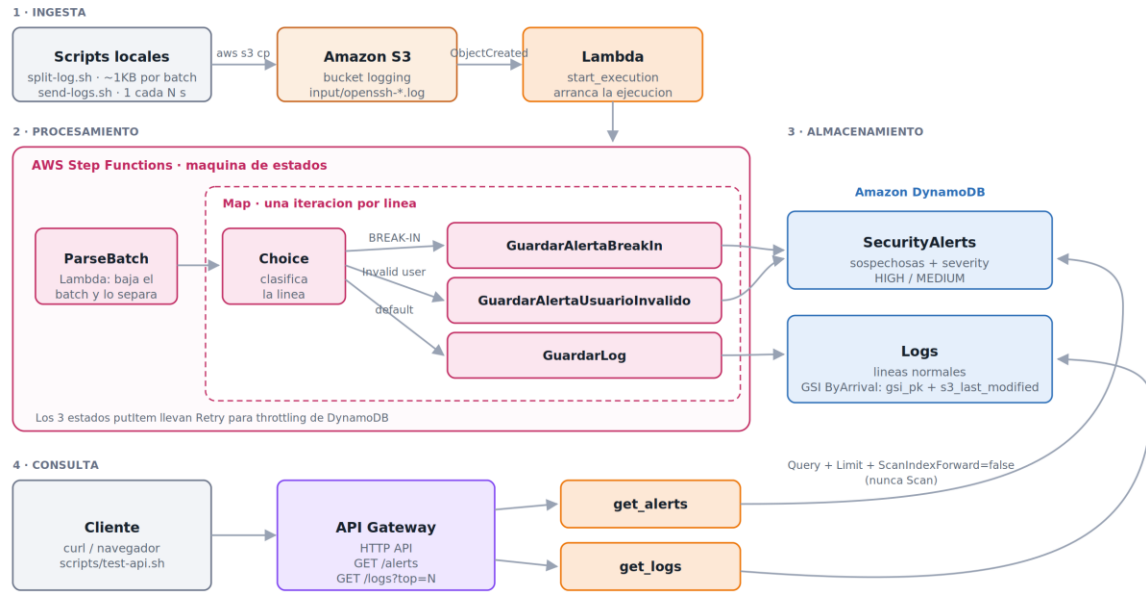
La ingesta arranca en local. split-log.sh lee el log línea por línea y agrupa líneas completas hasta superar 1 KB, sin partir ninguna a la mitad, por lo que cada batch queda apenas arriba de 1024 bytes. Luego start_logging.sh sube cada batch a input/ esperando N segundos entre uno y otro. El log es el dataset OpenSSH de loghub visto en clase.

Cada objeto que llega a input/ genera un evento ObjectCreated. Como S3 no puede invocar Step Functions directamente con una notificación de bucket, una Lambda mínima llamada start_execution traduce el evento y arranca la máquina de estados.

El procesamiento vive en Step Functions. ParseBatch invoca una Lambda que descarga el batch y lo separa en líneas, extrayendo timestamp, hostname, programa, PID y mensaje. Un estado Map las recorre y, para cada una, un estado Choice decide su destino: si contiene Invalid user o POSSIBLE BREAK-IN ATTEMPT va a SecurityAlerts con su severidad, y si no, a Logs. Los tres estados que escriben en DynamoDB llevan Retry con espera exponencial, de modo que un throttling reintenta solo esa línea en vez de hacer fallar toda la ejecución.

La consulta se expone con una HTTP API de API Gateway: dos rutas, cada una con su propia Lambda. GET /alerts regresa las alertas con id, timestamp, host, log y severity; GET /logs?top=N regresa los últimos N registros. Como las fechas del log son fijas y de 2017, el orden real de llegada se toma del LastModified del batch en S3, guardado en s3_last_modified, que es la sort key del índice ByArrival. Eso permite resolver los últimos N con Query, Limit y ScanIndexForward=false en lugar de un Scan.

DIAGRAMA DE ARQUITECTURA



Flujo completo: ingesta en S3, procesamiento en Step Functions, almacenamiento en DynamoDB y consulta vía API Gateway.

COMPONENTES

- **Amazon S3** - bucket logging. Recibe los batches en `input/` y emite el evento `ObjectCreated`.
- **Lambda start_execution** - traduce el evento de S3 y arranca la ejecución de la máquina.
- **Lambda parse_batch** - descarga el batch y lo separa en líneas individuales.
- **Step Functions** - máquina logging-system: Map, Choice y tres `putItem` con `Retry`.
- **DynamoDB SecurityAlerts** - líneas sospechosas con su `alert_type` y su `severity`.
- **DynamoDB Logs** - líneas normales, con el índice `ByArrival` por hora real de llegada.
- **API Gateway** - HTTP API con `GET /alerts` y `GET /logs?top=N`.
- **Lambdas get_alerts y get_logs** - una por endpoint; consultan el índice con `Query`.